Ders 2 Genişletilmiş Notu: Tasarım Kalıplarına Giriş, MVC ve Abstract Factory Kalıbı

Ders 2 Genişletilmiş Notu: Tasarım Kalıplarına Giriş, MVC ve Abstract Factory Kalıbı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tasarım Kalıbı Nedir?
- 2. Tasarım Kalıplarının Yararları
- 3. Kalıpların Sınıflandırılması ve Anlatım Sırası
- 4. Model-View-Controller (MVC) Kalıbı
- 5. Yaratımsal Kalıplar: Abstract Factory
- 6. Örnek Uygulama: 3B Oyun Motoru ve Üç Çözüm
- 7. Çıkartılan Dersler: Tasarıma Ait Genel Kurallar
- 8. Ders Sonundaki Soru-Cevaplar
- Örnek Sorular
- Dersin Toparlaması

1. Tasarım Kalıbı Nedir?

Bu derste Gang of Four'un (dörtlü çete) tanımladığı biçimde tasarım kalıplarına giriş yapılır.

1.1 Tanım ve amaç

Slayttaki tanıma göre (ekrandan, 00:20:00) tasarım kalıbı, yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir. Çözüm somut bir gerçekleme olmak zorunda değildir, soyut bir tarif de olabilir. Farklı problemler farklı ayrıntılar gerektirdiği için kalıp bir temel verir; tasarım mutlaka verilir, ama bazı kalıplarda çok ayrıntıya girilmez.

Tasarım kalıplarının temel amacı, tekerleği yeniden keşfetmek zorunda kalmadan "iyi" bir tasarıma ulaşmaktır. "İyi" kavramı geçen derste tartışıldı: problem hangi düzeyde parçalara ayrılabiliyorsa o düzeyde parçalara ayrılan çözüm iyi bir çözümdür ve iyiliğin en temel somut ölçütleri uyum (cohesion) ile bağlaşımdır (coupling). Bunun yanında sorunların esnek biçimde ve yeniden kullanılabilirliği artıracak yönde çözülmesi beklenir. Esneklik, kaçınılmaz olan gereksinim değişikliklerine tasarımın ve kodun kolay uyum sağlayabilmesidir.

Kalıplar esnek çözümler sunma iddiasındadır ve derste bu iddia örneklerle gösterilir. Bazı kalıplarda önce akla ilk gelen çözüm yapılır, sonra gereksinim değiştirilir ve bu çözümün sorunlu olduğu görülür. Ardından kalıp kullanmadan doğru bir nesneye dayalı çözüm, en sonda da kalıp ile çözüm incelenir ve getirileri ile götürüleri tartışılır. Bazı kalıplarda yalnız tek çözümle kalıbın uygulanması gösterilir.

Yeniden kullanılabilirlik de önemlidir. Hoca, çalışmaların yeniden kullanımın sağladığı tasarrufu sınırlı bulduğunu, ama yine de yeniden kullanımın "çoktan iyi" olduğunu söyledi.

1.2 Bir tasarım kalıbının dört bileşeni

Slayta göre (ekrandan, 00:20:00) her tasarım kalıbı dört ana bileşenden oluşur:

- **İsim:** Kalıbı en güzel şekilde anlatacak, kolay hatırlanmasını sağlayacak bir ya da birkaç kelimelik isim. Hocaya göre bir şeyi adlandırmak, o şey ne olursa olsun, çok kritiktir.
- **Sorun:** Kalıbın çözdüğü sorunun tarifi; bunun neden bir sorun olduğu.
- **Çözüm:** Sorunu çözmek için önerilen tasarımın bileşenleri, bu bileşenlerin ilişkileri, sorumlulukları ve birlikte çalışmaları. GoF kalıpları nesneye yönelik kalıplar olduğu için bileşenler sınıflardır; sınıfların üye alanları, metotları, sorumlulukların nasıl atanacağı ve nesnelerin nasıl davranacağı anlatılır.
- **Tartışma/Sonuçlar:** Çözümün sistemin geneline esneklik, genişletilebilirlik, taşınabilirlik gibi yönlerden etkileri; çözümün güçlü ve zayıf yönleri, yer-zaman çelişkileri ve başka kalıplar ile işbirliği olanakları.

1.3 Kalıp sihirli değnek değildir

Hoca iki noktanın altını çizdi. Birincisi, tasarım kalıbı sihirli değnek değildir; literatürde bu tür mucize çözümlere “silver bullet” (gümüş kurşun) denir. Kurt adam normal kurşunla değil gümüş kurşunla öldürülür; yazılımda böyle bir mucize yoktur.

Hocaya göre mühendisin temel tanımı hesap yapan kişidir. Ama bu hesap yalnız aritmetik ya da kalkülüs hesabı değildir: mühendis, problemi çözmek için kullanabileceği yolları, bileşenleri ve yöntemleri, hangisinin hangi tür problemde daha verimli olacağını ve hangi dezavantajları getireceğini hesaplayabilen kişidir.

Yer-zaman çelişkisi (time-memory trade-off) bilgisayar bilimlerinde bilinen bir kavramdır. İşlem gücünden tasarruf etmek isteyen, daha fazla bellek kullanabilir: örneğin bazı hesapların sonuçlarını önceden bir tabloda saklayıp gerektiğinde oradan okumak (look-up), karmaşık bir denklemi yeniden hesaplamaktan daha hızlıdır. Tersine, az bellek kullanmak için kod yazılırsa ara hesaplamalar artar, işlem uzun sürer ve işlemci daha çok kullanılır. İyi çözümler bunu en uygun noktaya getirmeye çalışır.

Kalıpların temel hitap ettiği alan modellemedir, çalışma zamanı başarımı değildir. Hatta bazı yerlerde kalıpların çalışma zamanı başarımına olumsuz etkisi olabilir; bu konu refactoring kısmında daha çok tartışılacaktır. Ama kalıbın sunduğu esnekliğin yanında bir işlemin 100 milisaniye yerine 200 milisaniyede yapılması bir kayıp sayılmaz; insan bu farkı algılamaz. Buna karşılık esneklik sayesinde gereksinimler çok kolay hayata geçirilir ve uzun, pahalı mesailer gerekmez.

İkinci nokta, bazı kalıpların bazı problemlerde birlikte çalışarak sinerji doğurabilmesidir. GoF bunları kalıpların anlatımında belirtir, ama bu ilişkiler bunlarla sınırlı değildir. Bir problemle karşılaşıldığında GoF’un önermediği yeni işbirlikleri keşfedilebilir; hiçbir şey yalnız kitapta yazıldığı gibi harfiyen geçerli değildir.

1.4 Kalıplar arasındaki olası ilişkiler

GoF kitabındaki bir şema kalıplar arasındaki olası ilişkileri gösterir (ekrandan, 00:23:00). Şemadaki ilişkilerden bazıları şunlardır:

- Command → Composite: “composed using” (komutlar Composite ile birleştirilir).
- Strategy → Flyweight: “sharing strategies”; State → Flyweight: “sharing states”.
- Composite → Iterator: “enumerating children”; Composite → Visitor: “adding operations”; Composite → Decorator: “adding responsibilities to objects”.
- Abstract Factory → Singleton: “single instance”; Abstract Factory → Factory Method: “implement using”; Abstract Factory → Prototype: “configure factory dynamically”.
- Facade → Singleton: “single instance”; Mediator ← Observer: “complex dependency management”.

Hoca şemanın nasıl okunacağını örneklerle anlattı. Command kalıbı, örneğin olay tabanlı bir GUI’de basit komutları ayrıntılı metotlara eşleştirmekle ilgilidir. Komutlar birden fazlaysa ya da birbirine zincirlenmesi gerekiyorsa, Composite kalıbı ile bir araya getirilebilir.

State kalıbı bir varlığın içinde bulunabileceği durumları ve bir durumdan diğerine geçişi modeller. Strategy kalıbı ise bir işin birden fazla yapılış yolu olduğunda çalışma anında en uygun olana geçebilmeyi modeller; iki kalıp dış tasarım olarak benzerdir. Bunların strateji ve durum nesneleri Flyweight olarak modellenebilir. Flyweight'te aynı türden çok sayıda varlık vardır. Her birinin durum bilgisini ayrı ayrı saklayıp bellekte çok yer harcamak yerine, veriler tüm örnekler arasında ortak olan ve farklılaşan kısım olarak ikiye bölünür. Ortak kısmı taşıyan nesneler bellekte saklanır; nesne kullanılırken değişen veri kısmı ona verilir.

Kitap strateji nesnelerinin Singleton da olabileceğini söyler, ama şemada bu ilişki çizilmemiştir. Demek ki şema yalnız bu ilişkilerin olabileceğini söyleyen kapalı bir liste değildir. Yeni problemlerle karşılaştıkça şemaya yeni olası etkileşimler eklenebilir.

2. Tasarım Kalıplarının Yararları

Slayttaki yararlar şunlardır (ekrandan, 00:30:00):

- Tekerleği yeniden icat etmemek.
- Üzerinde birçok deneyimli kişinin emeği geçtiğinden "kaliteli" bir çözüm.
- Deneyimin yazılı biçimde aktarılabilmesi.
- Yazılım ekibi arasında ortak bir sözlük oluşturması.

Hoca son iki yararın ikinci planda görülmemesi gerektiğini, çok önemli olduklarını söyledi.

2.1 Deneyimin yazılı aktarımı

GoF ve sonradan yeni kalıplar kazandıranlar nesneye yönelimde tecrübeli kişilerdir. Bu kişiler deneyimlerini belirli bir probleme uyarlayarak paylaşır. Deneyim normalde yazılı olarak pek aktarılmaz; usta-çırak ya da öğretmen-öğrenci ilişkisi içinde, gösterilerek ve yapılarak aktarılır. Tasarım kalıpları ise kullanıma hazır şablonlardır ve dört bölümlük yapılarıyla konuyu çok ayrıntılı anlatır. Bir kalıbı bir probleme uyguladığınızda o kalıbı öğrenmiş, onu ortaya çıkaran deneyimi almış ve içselleştirmiş olursunuz.

2.2 Ortak sözlük

Kalıpları bilen kişiler arasında çok daha etkili ve verimli bir iletişim kurulur. Hoca iki örnek verdi. Tasarım aşamasında iki kıdemli tasarımcı konuşuyor olsun. Biri "Burada farklı finansal algoritmalar olabilir, müşteriler farklı algoritmalar kullanmayı çok seviyor; bunu Strategy kalıbına göre tasarlarsak algoritmaları kolayca değiştirebilirler" der. Öteki "İyi fikir, ama bunları Singleton yapmayalım, aynı algoritmanın çok farklı varyasyonları olabiliyor; onun yerine Flyweight olabilir" diye cevap verir. İki cümlelik bir konuşmada tasarımın önemli bir kısmına karar verilmiş ve karşılıklı anlaşılmış olur.

Kodlama aşamasında da bir programcı bir kısmı verimli kodlayamamışsa, arkadaşı "Burada Composite kullansan bütün farklı nesneleri bir arada daha rahat tutabilirsin" der. Uzun uzun "şöyle tasarla, şöyle kodla" demek yerine tek kelimeyle kalıp adı söylenir ve öteki kişi gereğini yapar.

> **Hocanın vurgusu:** Ortak sözlük yararının önemini lütfen hafife almayın.

2.3 Tartışma: kalıp mı, basit ilke mi?

Slayttaki tartışma maddesine göre bir kişinin "kalıp" dediğine diğeri "basit bir yapı taşı" ya da "temel bir ilke" diyebilir. Hoca bu itirazın bazı kalıplar için haklı olduğunu kabul etti; bazı kalıplar gerçekten basit bir fikir düzeyinde anlatılır. Ama hocaya göre basit olsa da yararlı ve kullanışlı olduğu sürece tasarım kalıbı adını hak eder.

3. Kalıpların Sınıflandırılması ve Anlatım Sırası

3.1 Amaca göre üç grup

GoF kalıpları amaçlarına göre üç kümeye ayırır ve bu sırayla anlatır:

- **Yaratımsal (creational):** Nesnelerin oluşturulması, temsil edilmesi ve ilişkilendirilmesindeki bağımlılığı, yani bağlaşımı azaltmaya yönelik kalıplar. Bağlaşımın düşük olmasının tercih edildiği geçen derste konuşulmuştu.
- **Yapısal (structural):** Sınıfları ve nesneleri bir araya getirerek daha büyük yapılar elde etmeye yönelik kalıplar. Yalnız bir araya getirmeyi değil, bu ilişkilerin idamesi gibi yan problemleri de ele alırlar.
- **Davranışsal (behavioral):** Sorumlulukların nesnelere dağıtılmasına ve doğru algoritma seçimine yönelik kalıplar. Sorumlulukların doğru dağıtılması uyumun yüksek olmasını sağlar.

Hoca bu sıranın mantığını şöyle tahmin etti: oluşturulmamış şeyler kullanılamaz ve bir yapıda ilişkilendirilemez, bu yüzden önce yaratımsal kalıplar gelir. Sonra oluşturulan nesnelerin nasıl bir araya getirileceği (yapısal), en son da elimizdeki nesnelerin davranışlarının en esnek şekilde nasıl modelleneceği (davranışsal) anlatılır. GoF kitabında ilk anlatılan kalıp ise MVC'dir. MVC aslında davranışsal bir kalıptır; hocaya göre çok ayrıntılı kod içermediği için giriş olarak seçilmiş olabilir.

Hoca ekranda dersin kalıp listesini de gösterdi (ekrandan, 00:31:00). Sıra şöyledir: MVC (davranışsal); Abstract Factory, Builder, Factory Method, Prototype, Singleton (yaratımsal); Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy (yapısal); Chain of Responsibility, Command, Iterator, Memento, Observer, State, Strategy, Template Method ve diğer davranışsal kalıplar.

Bir problemde hem yaratımsal, hem yapısal, hem davranışsal ihtiyaçlar olabilir. Hoca bu gruplamanın bir ipucu olarak kullanılabileceğini söyledi: yaratıma yönelik bir kalıp uyguladıktan sonra, sistemdeki davranışı daha iyi modellemek için bir ya da birden fazla davranışsal kalıp da yararlı olabilir. Ya da nesneler arası ilişkilerdeki yapıyı iyileştirmek için yapısal kalıplar gözden geçirilebilir.

Hoca bunu bir günlük hayat benzetmesiyle anlattı. Bisikletiniz yokken etraftaki bisikletçi dükkanlarını görmezsiniz. "Bir bisiklet alsam hem spor olur hem güzel havalarda dolaşırım" diye düşünmeye başladığınızda etraftaki bütün bisiklet dükkanlarını görürsünüz. Kalıplara yeni başlandığında listeye bakıp "bu kalıp problemin bir köşesine uyar mı?" diye tek tek denemek yararlıdır, ama verimli bir yol değildir. Kalıplar iyi özümsenip çeşitli örneklerle uygulandıktan sonra, problem görüldüğünde hangi kalıbın yarayacağı doğrudan anlaşılır.

3.2 Anlatım sırası üzerine hocanın deneyimi

GoF bu üç gruplamaya ek olarak kalıpları çözdükleri problemlerin genel tarifine göre başka bir gruplamayla da ele alır. Hoca bir dönem kitaptaki sırayı değil bu alternatif gruplamayı kullanmayı denedi. Ancak bu sırada çevrimsel referans sorunu çıktı. Örneğin davranışsal Observer kalıbı anlatılırken, gözlemlenen ya da gözleyen varlıkların ağaç yapısında bulunabileceği ve bunun yapısal Composite kalıbıyla iyi modellenebileceği söylenir. Composite henüz görülmediği için anlatım havada kalır. Bu yüzden hoca kitaptaki sıraya döndü; konuyu anlatan diğer kaynakların hepsi de bu sırayı kullanır.

3.3 Sınıfsal ve nesnesel kalıplar

Slayta göre (ekrandan, 00:40:00) GoF kalıplarının bazılarının odağı sınıflar, bazılarınınki nesnelerdir:

- **Sınıfsal kalıplar:** Sınıflar ve alt sınıfları arasındaki ilişkilere yöneliktir. Sınıf yapısı ve kalıtım ilişkisinin doğası gereği bu ilişkiler derleme zamanında belirlenir ve daha durağandır.
- **Nesnesel kalıplar:** Çeşitli türden nesneler arasındaki ilişkilere yöneliktir. Kodun çalışması süresince rahatlıkla değiştirilebilen, daha dinamik ilişkilerdir; örneğin bir üye alana null ya da başka bir nesne atamak gibi basit setter işlemleriyle değiştirilir.
- Adapter kalıbı GoF tarafından hem sınıfsal hem nesnesel kalıp olarak değerlendirilmiştir. Diğer kalıplar bu iki gruptan yalnız birine dahildir.

Hoca kalıtımın durağanlığını bir örnekle açıkladı. İnsan sınıfından kalıtımla Çalışan sınıfı türetilirse her çalışan aynı zamanda bir insandır ve bu çalışma anında değiştirilemez. Kardeş sınıf Öğrenci olsun. İşten çıkan bir kişi öğrenci olarak modellenecekse, Çalışan nesnesi doğrudan

Öğrenci'ye dönüştürülemez, çünkü bunun için nesnenin sınıfının değişmesi gerekir. Ancak bir kopya kurucu (copy constructor) yazılıp eski nesnenin ortak insan bilgileri yeni Öğrenci nesnesine taşınabilir. Bu durumda nesnenin kimliği kaybolur: eski Çalışan nesnesi bellekte bir yerde, yeni Öğrenci nesnesi başka bir yerde durur.

Aynı ilişki sahiplik (parça-bütün) ilişkisiyle kurulsaydı, insan nesnesi sahip olduğu rolleri üye alanlarında tutardı. Kişi çalışırken "çalışan" üye alanına bir nesne atanır; işten çıkınca insanla ilgili bilgiler sabit kalır ve yalnız bu alan null yapılır. Öğrenci olunca "öğrenci" alanına yalnız öğrencilik bilgilerini taşıyan bir nesne atanır.

Hoca bu kırılımların ezberlenmesine gerek olmadığını söyledi. Bazı kitap ve sitelerdeki testlerde "dört kalıptan hangisi diğerlerinden farklıdır?" türünden sorular çıkabilir; ama derste kalıpların kategorileri zaten yazılı olarak verilir.

4. Model-View-Controller (MVC) Kalıbı

4.1 MVC ve Microsoft'un MVC çatısı

MVC denince, özellikle tasarım kalıpları üzerinde çalışmamış olanların aklına önce Microsoft'un MVC çatısı (framework) gelebilir. Hoca, Microsoft'un MVC çatısının bu tasarım kalıbına dayanan, çok kullanışlı ve çok daha geniş ölçekli bir yapı olduğunu, hakkının verilmesi gerektiğini söyledi. MVC'nin kaynağı olarak ise GoF'un tasarım kalıplarını gösterdi.

> **Ders sonrası not:** Hoca MVC'nin ilk kullanıldığı yeri GoF'un tasarım kalıpları olarak andı. MVC büyük olasılıkla daha eskidir: 1970'lerin sonunda Smalltalk ortamında ortaya çıkmıştır. GoF kitabı (1994) MVC'yi kendi kalıplarını tanıtmak için giriş bölümünde örnek olarak kullanır.

4.2 Sorun

Slayta göre (ekrandan, 00:53:40):

- Aynı veriyi farklı biçimlerde gösterebilmek istiyoruz.
- Kullanıcıya verinin nasıl ve ne kadarının gösterileceğini seçebilme gibi olanaklar sağlamak istiyoruz.

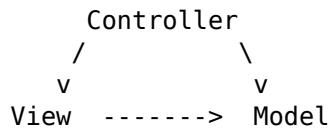
Kullanıcı verinin hepsini görmek zorunda değildir, bir kısmını görebilir; veriyi farklı biçimlerde görmek isteyebilir. Ayrıca kullanıcının veriyi değiştirebilmesi, isterse farklı biçimlerde değiştirebilmesi de istenir. Bütün bunları mümkün kılan bir altyapı nasıl modellenir? Kalıbın cevabı budur.

4.3 Çözüm: değişeni bul ve sarmala

Temel ilke şudur: **neyin değiştiğini bul ve onu sarmala.** Burada değişebilen şeyler veri, verinin gösterimi ve verinin gösteriminin yönetimi ile düzenlenmesidir. Bunlar değişebiliyorsa farklı sınıflar olarak ele alınmalıdır:

- **Model:** Ham veriyi simgeler.
- **View:** Verinin farklı gösterim biçimlerini simgeler.
- **Controller:** Gerek farklı View'lar arasında geçiş, gerek aynı View verinin farklı bölümlerini gösteriyorsa "şu an şu bölümü göster" gibi istekleri yürütür. Veride değişiklik yapma işlemleri ve View'ın ayarlarıyla ilgili değişiklikler de Controller'dadır.

Sınıflar arasındaki ilişkiler slayttaki şemadadır (ekrandan, 01:00:00):



Yani Controller → View, Controller → Model ve View → Model okları vardır. Model'den başka bir sınıfa giden ok yoktur.

Model veridir; kendisinin nasıl gösterildiğinden ve nasıl değiştirildiğinden bağımsız olmalıdır. Değiştirildiğinde yalnız "değerim değişti, yeni değer bu" bilgisine sahip olması yeterlidir; şu

yolla ya da bu yolla değiştirildiğini bilmesi gerekmez. Hoca bunu nesneye yönelimin önemli bir olgusuna bağladı: bir nesnenin metodu çağrıldığında, nesne o metodu çağıranın kim olduğunu bilmez. Belki çağıran, kendi referansını (this) parametre olarak iletebilir, ama bu da null olabilir. Madem nesne kendisini kimin çağırdığını bilmez, veri de kendisini kimin değiştirdiğini ya da gösterdiğini bilmek zorunda değildir.

View hangi veriyi göstereceğini bilmek zorundadır; bu yüzden View'dan Model'e bir sahiplik ilişkisi vardır. Controller hem veriyi doğrudan düzenleyebildiği için Model'e, hem de View ile ilgili ihtiyaçları alıp View'a ilettiği için View'a bağlıdır. Diğer yönlerde ok yoktur; böylece temel ihtiyaçlar kapsamında bağlaşım en düşük düzeyde tutulur.

4.4 Adın kaynağı: ressam benzetmesi

Kalıbın adı bir ressam ile çizdiği kişiye benzetilir. Sanat dünyasında resmi çizilen kişiye de, cansız nesnelere de "model" denir. Bir ressama resmimi çizdirmek istersem model benim. Ressama "benim gerçekçi bir portremi çiz" dersem, ortaya çıkacak resmin bakış açısını vermiş olurum: omuzdan yukarısı mı, tam boy mu? Bu "view"dur, beni gösterecek pencerenin ebadıdır. Belki "beni karikatür olarak, karakalem çiz, burnumu büyüt" de diyebilirim. Ama hangi tarafın ne kadar abartılacağı, nihai ürünün ayrıntıları ressama kalmıştır; ressam kontrolcüdür.

Hoca günümüz için bir de üretken yapay zeka benzetmesi yaptı: kendi resmini ya da vesikalık fotoğrafını yükleyip "beni süper kahraman gibi çiz", "şuradan yukarısını göster", "şurası çok büyük olmuş" diyerek tarifi çalışma anında değiştirmek de aynı ayrımdır. Bu benzetme akılda tutulduğunda kalıbın adı ve adından da tasarım ayrıntıları kolayca hatırlanır.

4.5 Yararlar ve Excel örneği

Slayta göre (ekrandan, 01:10:00) gösterim, kontrol ve veri birbirinden ayrılarak:

- Bunlar çalışma anında değiştirilebilir.
- İlgi alanlarının ayrılması sağlanır. Farklı sorumluluklar ayrıldığı için uyum da yükselir.
- Yeniden kullanım artar.

Hoca yeniden kullanımı Excel'de bir örnekle gösterdi (ekrandan, 01:02:30). Bir seçimin sonuçları veri olsun:

Parti	Oy (%)
A	38
B	27
C	15
Diğer	20

Toplam $38 + 27 + 15 + 20 = 100$'dür (Excel'in durum çubuğunda da "TOPLAM: 100" görünür). Bu veri tablo olarak, pasta grafiği ya da sütun grafiği olarak görülebilir. Hoca önce iki boyutlu bir pasta grafiği ekleyip veri etiketlerini gösterdi, sonra grafik türünü değiştirerek sütun grafiğine geçti. Uygulama değişip veri seçim sonuçları yerine katılımcıların demografik bilgileri (kadın, erkek, çalışan, çalışmayan vb.) olsa bile grafik çizme işleri, yani View'lar değişmez. Önceki uygulamanın View bileşenleri yeni uygulamada yeniden kullanılabilir.

4.6 Çeşitlemeler

Slayttaki çeşitlemeler şunlardır (ekrandan, 01:10:00 ve 01:30:00):

- Model'in geçirdiği değişiklikleri View'a bildirmesi.
- Birden fazla View: iç içe (nested), ayrı ya da tek görevcikte (thread), örtüşen ya da ayrık.
- Birden fazla kontrolcü: farklı denetim biçimleri (klavye, fare, ses, …).

Model'in View'a bildirmesi. Veriyi değiştirmenin tek yolu Controller mıdır, yoksa veriyi değiştiren başka bir kaynak da var mıdır? Seçim örneğinde sandıklar açıldıkça oranlar değişir. Veriler ilçe seçim kurullarından il seçim kurullarına, oradan Yüksek Seçim Kurulu'na bilgi sistemi üzerinden gider ve televizyon kanalları bunu kendi uygulamalarıyla gösterir. Grafiklerin

çizdiği veri sürekli değişir ve grafiklerin de güncellenmesi gerekir. Bu değişiklikten ilk haberi olan nesne verinin kendisi, yani Model'dir. Model, değiştiğinde View'a "artık farklı değerlerle çizilmem gerekiyor" diye bildirsin ve View yeni veriyi alıp kendini tazelesin isteniyorsa, Model'den View'a da bir ok çizilir. Bu durumda Model kendisini gösteren View ya da View'ların hangileri olduğunu bilmelidir.

Bu ok zorunlu değildir. View belli bir periyotta Model'i yoklayıp değişip değişmediğine bakabilir, değiştiyse kendini yeniden çizer. Yoklama aralığı senaryoya ve problem alanına göre değişir. Seçim gecesinde 10 saniyede bir bakmaya gerek yoktur, 10 dakikada bir de pek gerek yoktur, ama saat başı bakmak geç kalır; izleyiciler "bu kanal bir saat geriden geliyor" deyip başka kanala geçebilir. Bileşenler farklı sunucularda ise sık sorgulama ağ trafiğini ve haberleşme maliyetini artırır. Sık sorgulayan taze veriye ulaşır ama maliyet artar; maliyeti kısan bayat veri gösterme riskini alır. Hocaya göre burada karar "bağlaşıma mı oynayayım, başarıma mı?" sorusuna, hangisi daha önemliyse ona göre verilir. Mümkün olan en hızlı ve anlık güncelleme isteniyorsa tek seçenek Model'den View'a ek ilişkiyi kurmaktır.

> **Hocanın vurgusu:** Tasarım kalıplarına harfiyen, birebir uymak gerekmez. Kalıbın temel önerilerinde işimize gelecek şekilde değişiklik yapabiliriz; kalıp bir yasa ya da kanun değildir.
>
> **Ek bilgi (derste anlatılmadı):** Hocanın ekranındaki slayt notunda GoF kitabından şu cümle görünür: "The View-Controller relationship is an example of the Strategy (315) design pattern" (ekrandan, 01:30:00). Yani View'ın kullanıcı girdisine nasıl tepki vereceğini belirleyen algoritma, büyük olasılıkla değiştirilebilir bir Controller nesnesine verilmiştir. GoF kitabı büyük olasılıkla MVC'de Model-View ilişkisini Observer, iç içe View'ları ise Composite kalıbının örneği olarak da anar.

Birden fazla View. En basit örnek farklı grafik türleridir, ama daha karmaşık senaryolar da olabilir:

- **İç içe View:** Bütün Türkiye haritasında il düzeyinde toplam oy oranları gösterilir. İstanbul ya da Ankara gibi bir ile odaklanınca o ilin seçim bölgelerindeki oranlar ayrı ayrı görülebilir. Hiyerarşide yukarı çıkma ve aşağı inme işlemlerine göre grafikler düzenlenir. Kalıbın bunun gerçeklenmesi için özel bir önerisi yoktur, ama böyle bir durumun olabileceğini hatırlatır.
- **Farklı görevciklerde çalışan View'lar:** Hele View'ların veriyi değiştirme yeteneği de varsa (örneğin Excel'de grafikteki çubuğu sürükleyerek rakamı değiştirmek), farklı grafikler arasında veri tutarlılığını sağlayacak altyapı kurulmalıdır.
- **Örtüşen View'lar:** Borsada bir firmanın hisseleri hem BIST 100 hem Teknoloji 50 endeksinde yer alabilir. İki endeksin bir kesişim kümesi vardır, ama ayrı olan firmalar da vardır. Bunların farkında olunmalı ve gerekiyorsa ek önlem alınmalıdır.

Birden fazla View ya da Controller ve kalıtım. Farklı ve birden fazla View ya da Controller olacaksa kalıtım ilişkisiyle kardeş sınıflar kurmak yararlı olabilir. Örneğin varsayılan kontrolcü klavye ve fare ile düzenleme alıyor olsun. Sesle de kontrol istenirse, sesle çalışan kontrolcü klavye-fare kontrolcüsünün bütün yaptıklarını yapar, ek olarak ses komutlarını alıp işleyen üye alan ve metotlara sahip olur; bu yüzden onun alt sınıfı olabilir. Sanal gerçeklik başlığıyla havada sürükleme gibi başka bir girdi biçimi de eklenebilir. View için de aynı şey geçerlidir: View'dan PieChart ve BarChart alt sınıfları türetilir. Sistemde setView metoduna bir PieChart nesnesi verilirse pasta grafiği, BarChart nesnesi verilirse çubuk grafiği görünür. Hoca bu şemayı ekranda elle çizdi ve dörtgenleri sınıf olarak düşünmeyi istedi.

5. Yaratımsal Kalıplar: Abstract Factory

Aradan sonra üç temel kategoriden ilki olan yaratımsal kalıplara geçildi. Bu gruptaki ilk kalıp Abstract Factory'dir.

5.1 "Factory" kelimesi üzerine

Bir kalıbın adında "Factory" kelimesi geçiyorsa o kalıp nesne oluşturulmasıyla ilgilidir. Ama GoF'un önerdiği, adında Factory geçen iki ayrı kalıp vardır: Abstract Factory ve Factory Method.

> **Hocanın vurgusu:** Hiçbir zaman yalnız “Factory” deyip geçmeyin. Hangi kalıbı kastettiğinizi net olarak belirtin.

5.2 Amaç, örnek ve sorun

Slayta göre (ekrandan, 01:31:10):

- **Amaç:** Birbirleri ile ilişkili ya da aynı aileden olan nesneleri, sınıflarını belirtmeden oluşturabilmek.
- **Örnek:** Java’da çeşitli GUI bileşenlerinin görünüşünün (look-and-feel) çalışma anında değiştirilebilmesi; farklı Linux dağıtımlarında GNOME, KDE gibi pencere yöneticilerinden herhangi birini kullanarak çalışabilmenin gerekmesi.
- **Sorun:** Taşınabilirliği sağlamak için, istekçinin hangi tür görünüşün hangi sınıftan nesnelerle gerçeklendiğini bilmemesi gerekir.

Amaç cümlesi ilk bakışta kafa karıştırabilir: bir sınıftan nesne oluşturmak için kurucu (constructor) metodu çağırmak gerekir ve kurucunun adı sınıfın adıyla aynıdır. Sınıf adını belirtmeden nasıl nesne oluşturulur? Kalıp bunu, kalıtımdaki yerine geçebilirlik ilkesini kullanarak akıllıca çözer.

> **Ek bilgi (derste anlatılmadı):** Yerine geçebilirlik ilkesi literatürde büyük olasılıkla Liskov yerine geçme ilkesi (Liskov Substitution Principle) olarak bilinir: bir üst sınıf türünden nesne beklenen her yerde, onun alt sınıflarından bir nesne kullanılabilmelidir ve program doğru çalışmaya devam etmelidir. Abstract Factory’de istemci soyut fabrika ve soyut ürün türleriyle çalışır; ona verilen somut fabrika ve somut ürünler bu ilke sayesinde üst türlerin yerine geçer.

Hoca örnekleri güncelledi. Java’da Swing gibi kendi GUI oluşturma yöntemleri kullanılırken uygulamanın dış görünüşü (look-and-feel) çalışma anında değiştirilebilir; klasik, Motif, Metal gibi türleri vardır ve görünüş değişince bütün GUI bileşenlerinin teması değişir. Windows’taki temalar da benzerdir: bir tema seçildiğinde masaüstü arka planından pencere başlık çubuklarının rengine kadar bütün bileşenler bir paket halinde değişir. Uygulama “bana bir düğme ver” der, ama o düğme look-and-feel’e ya da pencere yöneticisinin düğme gerçeklemesine göre farklı sınıflardan olabilir. Temel problem, mümkün olduğu kadar taşınabilir kod yazmaktır.

5.3 GoF şemaları ve UML uyarısı

Hoca, derste gösterilen kalıp şemalarının doğrudan GoF kitabından alındığını belirtti (kitabın elektronik sürümünden ekran görüntüsüyle slayta eklenmiştir). Kitap hazırlandığında UML henüz bir standart olarak önerilmemişti. Farklı ekollerin farklı modelleme teknikleri ve şema kuralları vardı; UML daha sonra bunların güçlü yönlerini alıp bir araya getirdi. Bu yüzden GoF kitabından alınan sınıf şemaları UML standartlarına uygun görünmez. Hoca farkları şemanın üzerinde çizerek gösterdi:

- **Soyut sınıf:** GoF şemasında WidgetFactory soyut bir sınıftır, çünkü adı italik yazılmıştır. Ama küçük bir pencerede ya da yakından bakmadan italik yazı fark edilmeyebilir. UML bunun için stereotip özelliğini getirmiştir: sınıf adının hemen üstüne, aynı kutunun içine «`abstract`» yazılır.
- **Soyut metot:** İtalik yazılan metotlar da soyuttur. UML’de her metodun yanına `{abstract}` yazılır.
- **Dönüş türü:** GoF şemasında metotların dönüş türü gösterilmemiştir. UML’de dönüş türü metot parantezinden sonra iki nokta ile yazılır.
- **Kalıtım oku:** Hocaya göre en uyumsuz olan budur. GoF şemasında kalıtım üçgeni üst sınıftan aşağıda, alt sınıflara giden çizgilerin birleştiği yerde çizilir. Bu, veritabanı varlık-ilişki (ER) şemalarındaki karga ayağı (crow’s foot) gösterimine benzer. UML’de okun ucu üst sınıfa değecek biçimde, yukarıda çizilir.

  > **Hocanın vurgusu:** Kalıtım okunu bu eski biçimde çizmek, UML sınıf şeması çizme kurallarını bilmemenin ya da bu kurallara çalışmamanın en bariz göstergesidir. Bunu GoF için söylemiyorum, çizildiği tarihte gösterim öyleydi. Ama şimdi gösterim değişti; şemalarınızı mutlaka güncel UML kurallarına göre çizin.

Hoca, GoF şemalarını güncel UML standardına göre sıfırdan yeniden çizmediğini, bu uyarıyı sözlü olarak yaptığını belirtti. Derste kendi şemalarını Violet UML aracıyla çizdi; bu araçta soyut sınıflar «Abstract» stereotipi ve soyut metotlar {abstract} ile gösterilir (ekrandan, 02:10:00).

5.4 Örnek çözüm: WidgetFactory

GoF'taki örnek çözüm şöyledir (ekrandan, 01:40:00):

```
Client ----> WidgetFactory {soyut}
                CreateScrollBar() {soyut}
                CreateWindow()    {soyut}
              ^ (kalıtım)
     +---------+-----------+
MotifWidgetFactory     PMWidgetFactory
  CreateScrollBar()      CreateScrollBar()
  CreateWindow()         CreateWindow()

Client ----> Window {soyut}       <- PMWindow, MotifWindow
Client ----> ScrollBar {soyut}    <- PMScrollBar, MotifScrollBar

MotifWidgetFactory - - -> MotifWindow, MotifScrollBar  (oluşturur)
PMWidgetFactory    - - -> PMWindow, PMScrollBar        (oluşturur)
```

İstemci (client), sürükle-bırak yöntemiyle çeşitli GUI bileşenleri yerleştirilecek bir formdur: bir pencere, bir kaydırma çubuğu, bir düğme, bir metin kutusu çizilmek istenir. Kitap eski olduğu için örnekler eski Unix pencere yöneticilerinden verilir: PM (Presentation Manager) ve Motif. Güncel karşılıkları Linux için GNOME ve KDE olabilir: GnomeWindow, KdeWindow gibi kardeş alt sınıflar.

İstemcinin, pencerenin Presentation Manager'a mı, Motif'e mi, GNOME'a mı, KDE'ye mi ait olduğunu bilmesi gerekmez. Tek istediği kendisine bir pencere verilmesidir; ayrıntılarla uğraşmak istemez. Bu yüzden isteklerini yönlendirebileceği bir widget fabrikası vardır (eski terminolojide GUI bileşenlerine "widget" denir). İstemci, sahiplik ilişkisiyle bağlı olduğu WidgetFactory nesnesinden CreateScrollBar() ile bir kaydırma çubuğu ister. WidgetFactory, Window ve ScrollBar soyuttur (italik yazılmışlardır); yalnız alt sınıflar somuttur.

Her pencere yöneticisi için ayrı bir fabrika alt sınıfı yazılır: MotifWidgetFactory, PMWidgetFactory, GnomeWidgetFactory, KdeWidgetFactory gibi. MotifWidgetFactory'nin CreateScrollBar() metodu somut bir MotifScrollBar, CreateWindow() metodu somut bir MotifWindow nesnesi döndürür. Bu somut sınıflar Window ve ScrollBar'ın alt sınıflarıdır.

İstemci ile aradaki ilişkilerin hepsi soyut üst sınıflar türünden kurulmuştur. İstemciye bir MotifWidgetFactory nesnesi verildiğinde, CreateScrollBar() çağrısı aslında bir MotifScrollBar nesnesi döndürür. Ama istemci bunun MotifScrollBar olduğunu bilmek zorunda değildir; yalnız soyut ScrollBar'ın erişim metotları üzerinden alttaki somut nesneye ulaşır. Böylece istemci tam olarak hangi sınıftan nesneyle uğraştığını bilmez.

Yeni pencere yöneticileri çıktıkça yeni kardeş fabrika sınıfları ve o yöneticideki GUI bileşenlerini modelleyen yeni sınıflar yazmak, çatıyı genişletmek için yeterlidir.

5.5 Alternatif: switch-case ile fabrika

Soru: Bu başka şekillerde kodlanamaz mıydı? **Cevap:** Kodlanabilirdi. Örneğin değerleri farklı aileleri simgeleyen bir enum türü tanımlanır. Basit bir setter metoduyla enum değeri verilerek fabrikanın o türden nesneler üretmesi sağlanır. Bunu gerçeklemek için bütün create metotlarında bir switch açılır ve her olası aile için "case PM ise PM nesnesi döndür" gibi case'ler yazılır.

Dezavantajı nedir? Daha uzun kod yazmak başlı başına bir dezavantajdır, ama asıl sıkıntı yeni bir pencere yöneticisi geldiğinde ortaya çıkar. Enum'a yeni değer eklenir, switch'lerin geçtiği her yere yeni case ve nesne oluşturma kodu yazılır; bunun için mevcut fabrika sınıfının kaynak kodu değiştirilir. Güncelleme müşterilere aktarılırken değişen ve değişmeyen bütün sınıflar

birlikte aktarılmak zorunda kalınır. Kalıbın önerdiği tasarımda ise yalnız yeni eklenen sınıfların kaynak kodları iletilir, çünkü mevcut hiçbir şey değişmemiştir, yalnız yenilikler eklenmiştir.

> **Hocanın vurgusu:** Kod yazarken kendinizi switch-case kodlarken görürseniz, büyük olasılıkla o noktada bir kalıtım hiyerarşisi ve çok biçimlilik kurmanız gerektiğinin işaretidir. Tasarımınızı gözden geçirin: kalıtım ya da kalıtım yerine uygun bir parça-bütün ilişkisi kurun.

Hoca başka çözümler de olabileceğini, ama her birinin kritik bir dezavantajı olacağını söyledi; avantajlı bir çözüm akla gelirse paylaşılmasını istedi.

5.6 Genel kalıp yapısı

Problemden bağımsız genel yapı şöyledir (ekrandan, 01:50:00):

```
Client ----> AbstractFactory {soyut}
                CreateProductA() {soyut}
                CreateProductB() {soyut}
              ^ (kalıtım)
     +---------+-----------+
ConcreteFactory1       ConcreteFactory2
  CreateProductA()       CreateProductA()
  CreateProductB()       CreateProductB()

Client ----> AbstractProductA {soyut}  <- ProductA1, ProductA2
Client ----> AbstractProductB {soyut}  <- ProductB1, ProductB2

ConcreteFactory1 - - -> ProductA1, ProductB1  (oluşturur)
ConcreteFactory2 - - -> ProductA2, ProductB2  (oluşturur)
```

Soyut bir fabrika sınıfı (AbstractFactory) vardır ve üreteceği her ürün için bir create metodu bulunur. Hangi ailelerden ürün oluşturmak isteniyorsa, o ailelerin oluşturucu (fabrika) sınıfları AbstractFactory'den kalıtımla türetilir. İstemci istediği nesneleri aileye özel türler cinsinden değil, genel bileşen türü cinsinden ister. Soyut ürün sınıfları (AbstractProductA, AbstractProductB) vardır; bütün ailelerin ilgili üyeleri bunların somut alt sınıfları olarak kodlanır. Yeni aile için ek kardeş ürün sınıfları ve ek kardeş fabrika sınıfı oluşturulur; bu, kodun başka hiçbir yerine dokunmadan yapılır.

Katılımcıların görevleri:

- **AbstractProductX:** X ürünlerinin arayüzünü tanımlar. Örnekte Window: yer değiştirme, boyut değiştirme, başlık ve simge değiştirme gibi işlemler. Düğme ve metin kutusu da başka ürünlerdir.
- **AbstractFactory:** Ürünleri oluşturmak için gereken komutların imzalarının bulunduğu soyut üst sınıftır.
- **ConcreteFactory:** Belirli bir ailenin ürünlerini oluşturan somut fabrikadır.

5.7 Kalıbın uygulanabileceği anlar

Slayta göre (ekrandan, 01:51:50):

- Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğu, birleştirildiği ve sunulduğundan bağımsız olmasının istendiği durumlar.
- Bir sistemin farklı ailelerden herhangi birindeki nesneler tarafından yapılandırılması gerektiğinde.
- İlişkili ürün ailelerinden oluşan nesnelerin birlikte kullanılması gerektiğinde. Hoca, kendi deyişiyle "biraz uyduruk" bir örnek verdi: üyeleri farklı pencere yöneticilerinden gelen bir pencere listesi.
- Bir ürün ailesindeki nesnelerin bir sınıf kütüphanesi olarak sunulacağı, ancak gerçeklemeleri yerine yalnızca arayüzlerinin açığa çıkarılacağı durumlarda.

Burada "arayüz", Java'daki interface kavramı değil, bir nesnenin public metotlarının tümünün imzaları anlamındadır.

Hoca bu maddelerin bir kısmını GoF'tan tercüme ettiğini, bir kısmını aklına gelen ihtiyaçlardan eklediğini söyledi. Bunları ezberlemek ya da buradan yola çıkmak çok mantıklı olmayabilir; kalıplar kullanıldıkça ve akılda kaldıkça hangi problemde kullanılacakları kendiliğinden anlaşılır. Kelimeler bazen kafa karıştırıcı olabilir.

6. Örnek Uygulama: 3B Oyun Motoru ve Üç Çözüm

6.1 Çözülecek problem

Slayta göre (ekrandan, 01:56:00):

- Bir 3B oyun motoru hazırlıyoruz.
- Kullanılacak işletim sistemi ve ekran kartına göre OpenGL veya DirectX kütüphanesinin renderer, shader vb. bileşenleri kullanılacak.

Slayta göre örneğin kaynağı hocanın kendisidir. Oyun motorunun mümkün olduğu kadar geniş bir platforma hitap etmesi istenir. PC platformu kendi içinde farklı işlemcilere ve oyunlar için daha önemli olan farklı mimarilerde grafik kartlarına sahiptir. En yüksek başarımla çalışması istenen bir oyun Microsoft'un DirectX kütüphanesi kullanılarak hazırlanır. Ama DirectX, Microsoft'un kendi platformuna özel, kapalı kaynaklı bir kütüphanedir; Mac'lerde ya da Linux'ta yoktur. Linux'ta çalışmak için açık bir standart olan OpenGL üzerinden üç boyutlu işlemler yürütülür; ekran kartlarının Linux'ta da OpenGL sürücüleri bulunur.

Oyun motorundaki bazı işlemler:

- **Rendering:** Üç boyutlu bir cismin ekrana çizilmesi.
- **Shading (gölgelendirme):** Sahnede bir ya da birden fazla ışık kaynağı vardır; bunlardan gelen ışıkla ortamdaki varlıkların gölgelerinin nereye ve nasıl düşeceğinin hesaplanması.
- **Texture mapping (doku kaplama):** Üç boyutlu bir nesnenin dış yüzeyinin belirli bir dokuyla kaplanması. Örneğin oyuncu yalnız göğsünü kaplayan bir zırh giyiyorsa göğüs kısmı zırhla, kollar çıplak çizilir; insan nesnesinin farklı kısımları farklı kaplamalarla render edilir.

6.2 Çözüm 1: switch-case kullanımı

Hocanın akla ilk gelen çözümü enum ve switch-case oldu. Slayttaki sınıflar şöyledir (ekrandan, 02:00:00):

```
public enum LibraryType {
    OpenGL, DirectX;
}
public class OpenGLRenderer {
    public void renderOpA() { }
    public void renderOpB() { }
    public void renderOpC() { }
}
public class OpenGLShader {
    public void shadeOpA() { }
    public void shadeOpB() { }
    public void shadeOpC() { }
}
public class DirectXRenderer {
    public void renderOpA() { }
    public void renderOpB() { }
    public void renderOpC() { }
}
public class DirectXShader {
    public void shadeOpA() { }
    public void shadeOpB() { }
    public void shadeOpC() { }
}
```

Aynı slayttaki sınıf şemasında ApControl sınıfı bu dört sınıfın hepsine bağlıdır. Altyazıya göre ApControl'ün (ana oyun motoru sınıfı) yapısı şöyledir. OpenGLRenderer türünden bir üye alan, OpenGLShader türünden bir üye alan, DirectXRenderer ve DirectXShader türünden birer üye alan vardır; yani motor hem OpenGL hem DirectX platformunun render, shader gibi yeteneklerini kullanacak nesneleri tanımlar. Hangi kütüphaneyle çalışıldığı LibraryType türünden `library` adlı bir üye alanda saklanır. Kurucuda hangi ekran kartına sahip sistemde çalışıldığına göre ilklendirme kodları yer alabilir. doRendering metodunda bir switch ile `library` değerine bakılır: OpenGL ise OpenGL renderer nesnesinin render işlemleri, DirectX ise DirectX renderer nesnesinin render işlemleri çağrılır. doShading metodu da aynı biçimde switch-case ile shader nesnelerinden birini kullanır.

Bu çözüm işe yarar. Ama sorun gereksinim değiştiğinde ortaya çıkar.

6.3 Çözüm 1'in zayıf yönleri

Diyelim ki motor DirectX 10 ile çalışıyordu ve Microsoft ek yetenekler ve başarım iyileştirmeleri getiren DirectX 11'i çıkardı. Geriye doğru uyumluluk için hem DirectX 10 hem DirectX 11 desteklenmek isteniyor. Hoca, Violet UML'de şemayı açarak yapılması gerekenleri gösterdi:

- DirectX 11 için yeni DirectX11Renderer ve DirectX11Shader sınıfları eklenir. Texture mapping de yapılıyor olsaydı bir DirectX11Mapper da gerekirdi.
- Enum'a DirectX11 değeri eklenir.
- ApControl'e DirectX 11 renderer ve shader türünden yeni üye alanlar eklenir.
- doRendering ve doShading metotlarındaki switch'lere yeni `case DirectX11` eklenir ve yeni nesnelerin işlemleri çağrılır.

Yani yeni bir 3B çatı eklemek için uygulamanın her yerine müdahale edilir. Yarın DirectX 12, DirectX 13 gelecek; belki motor konsollara da uyarlanacak (PlayStation, Nintendo gibi platformların kütüphaneleri ayrıdır). Texture mapping, ray tracing, çarpışma algılama gibi her yeni işlem için de ayrı switch'ler gerekecek. Hocaya göre bu doğru bir yol değildir.

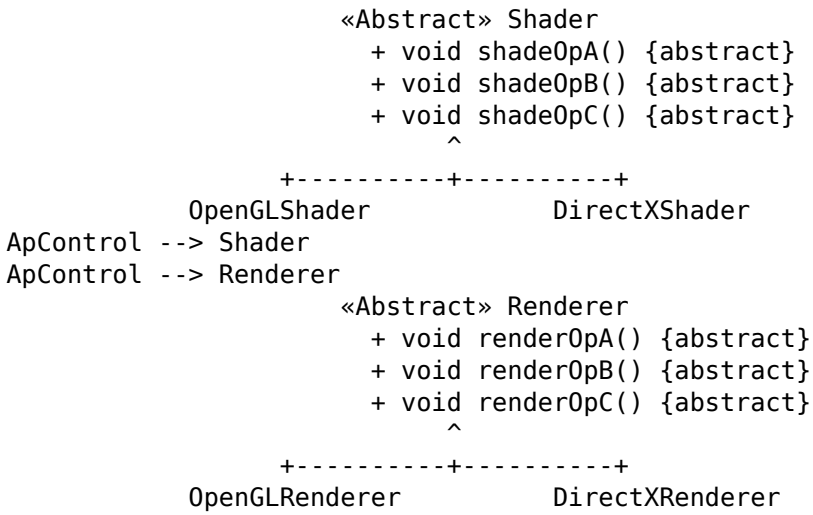
Slayttaki özet (ekrandan, 02:08:00):

- **Çözümün zayıf yönleri:** Yeni bir kütüphane eklemek için (örneğin DirectX 10, 11 ve 12 için ayrı kütüphaneler) çok fazla yerde kod değişikliği gerekir.
- **Tasarım ipucu:** Switch-case kullanımı, çok biçimliliğe gereksinim duyulduğunun ve/veya sorumlulukların atanmasında yanlışlık olduğunun uyarı işareti olabilir.

6.4 Çözüm 2: kalıtım ve çok biçimlilik

Abstract Factory'yi henüz bilmediğimizi varsayalım. İlk tasarımın doğru olmadığını, daha iyi bir kalıtım hiyerarşisi ve çok biçimlilik gerektiğini anladık. "Neyin değiştiğini bul ve onu sarmala" ilkesine göre farklı türde shader'lar ve renderer'lar olacak; yani değişen şeyler bunlardır ve birer sınıf hiyerarşisine alınmalıdır.

Violet UML'deki şema şöyledir (ekrandan, 02:10:00):



Soyut Shader sınıfında gereken gölgelendirme işlemleri (shadeOpA, shadeOpB, shadeOpC) soyut metot olarak yer alır. OpenGL ile çalışan bir shader yazmak için somut OpenGLShader sınıfı Shader'dan türetilir ve bu işlemlerin gövdeleri OpenGL ile kodlanır. Aynısı DirectXShader için yapılır. Renderer için de soyut Renderer sınıfı ve ondan türeyen OpenGLRenderer ile DirectXRenderer vardır.

Hoca kodu Eclipse'te gösterdi. Artık doRendering ve doShading içinde switch yoktur; ApControl soyut Renderer ve Shader türünden üye alanlar üzerinden çağrı yapar. Switch-case ise yalnız kurucuda, kütüphane türüne göre uygun somut nesneleri oluşturmak için kalır. Kütüphane türü, örneğin sistemin ekran kartına göre ya da bir yapılandırma dosyasından okunarak belirlenebilir. Yani switch-case'ten tamamen kurtulunmaz, ama yalnız bir yerde kodlanır. Çözüm 1'de ise rendering, shading ve sonra gelecek her işlem (texture mapping, ray tracing, çarpışma algılama) için ayrı switch'ler tekrarlanacaktı.

Hoca bu çözümde küçük bir hata buldu: `library` alanı varsayılan kurucuda hiç ayarlanmıyor, bu yüzden kurucudaki switch'in hiçbir case'ine girilmez. Kurucu, kütüphane türünü parametre olarak alıp ayarlayacak biçimde yazılmalıydı. Hoca derleyicinin bu konuda uyarı bile vermemesini ilginç buldu ve düzeltilmesi gerektiğini söyledi.

Hoca şemadaki gösterimle ilgili iki düzeltme de not etti: üye metotlar UML'de + ile public gösterilir, ama void dönüş türü metot adının önüne yazılmaz; dönüş türü varsa parantezden sonra iki nokta ile yazılır. Bir metodun sonunda gereksiz bir noktalı virgül de kalmıştı.

Bu tasarımda yeni bir kütüphane eklemek "tak-çalıştır" biçimindedir: yalnız yeni sınıflar kodlanır ve mevcut tasarıma eklenir. Mevcut kodda değişen tek yer kurucudaki ilklendirmeye eklenen yeni case'dir.

6.5 Çözüm 3: Abstract Factory kullanımı

Abstract Factory öğrenildikten sonra kalıp şöyle uygulanır. Soyut Shader ve soyut Renderer yine vardır, ama bunlar ya OpenGL ailesinin ya da DirectX ailesinin üyeleridir. Bir ComponentFactory sınıfı tanımlanır ve renderer, shader vb. nesnelerin oluşturulması için create metotları burada yer alır. Sonra her aile için bir somut alt sınıf oluşturulur: OpenGL ailesinin üyelerini döndüren OpenGLFactory ve DirectX ailesinin üyelerini döndüren DirectXFactory. Bu sınıflar üst sınıftaki soyut metotları somutlaştırır ve kendi ailelerinin üyelerini döndürür.

Eclipse'teki ComponentFactory sınıfı (ekrandan, 02:20:00):

```
package dp.abstractFactory.solution3;

public abstract class ComponentFactory {
    public abstract Renderer createRenderer();
    public abstract Shader createShader();
}
```

Eclipse'teki ApControl sınıfının görünen kısmı (ekrandan, 02:21:00):

```
package dp.abstractFactory.solution3;

public class ApControl {
    private ComponentFactory library;
    private Renderer r;
    private Shader s;
    public ApControl( ComponentFactory library ) {
        this.library = library;
        /* Seçenek A: library nesnesini ilklendirerek kütüphane türünü
         * switch-case ile önceki örnekteki gibi belirle
         * Seçenek B: buradaki gibi kurucu metoda parametre olarak ver.
         * Seçenek C: Farklı tür hedef bilgisayarlar için
         * farklı dağıtımlar yap ve alttaki uygun satırın yorumunu kaldır.
        // library = new OpenGLFactory();
        // library = new DirectXFactory();
        */
        r = this.library.createRenderer();
```

```
        s = this.library.createShader();
    }
    void doRendering ( ) {
        r.renderOpA();
        r.renderOpB();
        // ... (kare burada kesiliyor)
```

Paket gezgininde solution3 paketinde ApControl, ComponentFactory, DirectXFactory, DirectXRenderer, DirectXShader, LibraryType, OpenGLFactory, OpenGLRenderer, OpenGLShader, Renderer ve Shader dosyaları görünür (ekrandan, 02:20:00).

`library` alanı artık ilgili soyut fabrika türündendir ve kurucuya parametre olarak verilir; buraya gelen nesne elbette somut alt sınıflardan birine ait olacaktır. Renderer ve shader fabrikanın create metotlarıyla oluşturulduğu için switch-case ortadan kalkar; oyun motoru kodunda hiç switch-case kalmaz.

Oyun motorunu ayağa kaldırırken parametre olarak DirectXFactory mi yoksa OpenGLFactory nesnesi mi verileceğine karar veren bir ilk yapılandırma kodu bir yerde yine bulunmak zorundadır ve switch-case orada kalır; hangi seçeneklere karşılık hangi nesnenin verileceğine bakmak gerekir. Ama bu kod oyun motorunun içinde değil, motorun yapılandırma ön yüzünde yazılır. Böylece iş mantığı açısından sistem altyapısının ayrıntılarını bilmek zorunda kalmadan oyun motoruna odaklanılabilir.

Soru (hocanın kendi sorusu): Bu numara, yani fabrika nesnesini doğrudan vermek, Çözüm 2'de de yapılamaz mıydı? **Cevap:** Tam olarak yapılamaz. Çözüm 2'de yalnız bir enum ve iki ayrı nesne vardır; enum değerine bakıp "bu türse şu renderer'ı ve şu shader'ı oluştur" diyen bir switch-case (ya da aynı mantıktaki if-else if zinciri) mutlaka gerekir. Fabrika nesnesini doğrudan vermek mümkün değildir. Çözüm 3'ün Çözüm 2'ye üstünlüğü, switch-case'in ana odaklanılan sınıfların hepsinden uzaklaştırılmasıdır. Hoca, Çözüm 2'nin kurucusuna parametre vermenin çok fazla kazandırmadığını, ama yine de verilmesi gerektiğini, çünkü mevcut hâlinin bir hata olduğunu ekledi.

Hoca, kalıbın bu kullanımının ileride işlenecek bir refactoring eyleminin uygulanmış hâli gibi olduğunu, refactoring konusu işlendiğinde bu örneğe geri dönülebileceğini söyledi.

6.6 Çözümlerin karşılaştırılması

Çözüm 2'den Çözüm 3'e geçiş nispeten yumuşak oldu: başka hiçbir değişiklik gerekmedi. Sıfırdan kodlanıyorsa zaten doğrudan bu tasarıma göre kodlanır. Hocaya göre bu yumuşak geçiş, Çözüm 2'nin gerçekten de nesneye yönelimin ilkelerine ve önerilerine uyan bir çözüm olduğunun göstergesidir.

Peki ne kazanıldı, ne kaybedildi?

- **Sınıf sayısı:** Çözüm 3'te Çözüm 2'ye göre üç ek sınıf vardır (ComponentFactory, OpenGLFactory, DirectXFactory). Sınıf sayısı açısından Çözüm 1 en azdır, ama yeni kütüphane eklemede büyük sorun çıkardığı için en avantajlı değildir.
- **Yeni kütüphane eklemek:** Çözüm 2'de kardeş alt sınıflar (DirectX11Renderer, DirectX11Shader) eklenir ve kurucuya bir case eklenir. Çözüm 3'te aynı kardeş alt sınıflar ve ek olarak bir DirectX11Factory sınıfı eklenir; yani bir sınıf fazla yazılır, ama mevcut koda dokunulmaz.
- **Switch-case:** Çözüm 1'de her işlemde, Çözüm 2'de yalnız kurucuda, Çözüm 3'te ana sınıfların hiçbirinde yoktur (yalnız yapılandırma ön yüzünde).

Ek sınıfların yönetimine değip değmeyeceğine projenin boyutuna ve vakaya göre karar verilir.

> **Hocanın vurgusu:** Yeter ki birinci çözümü kullanmayın. İkinciyi ya da üçüncüyü kullanın. İkisi de birinciye göre daha fazla sınıfla uğraşır, ama çok daha esnektir; bakım masrafı daha düşük, geliştirmesi daha kolay çözümlerdir.

7. Çıkartılan Dersler: Tasarıma Ait Genel Kurallar

Slayttaki genel kurallar şunlardır (ekrandan, 02:30:00):

- **Neyin değiştiğini bul ve onu sarmala.**
  - Böylece bu şeyi soyutlamış olursun.
  - Böylece bu şeyi kara kutu yaklaşımı ile kullanabilirsin.
  - Böylece bu konu ile ilgili değişiklikler sadece bu kısmı etkiler.
  - Örnekte kullanılan yer: Hangi grafik kütüphanesinin kullanılacağı bir bilgisayardan başka bilgisayara geçilince değişeceğinden, bu değişim ComponentFactory sınıfında soyutlanmıştır.
- **Kalıtım ilişkisi yerine parça/bütün ilişkisini tercih et.**
  - Kalıtım farklı, sakıncalı bir biçimde kurulabilirdi: ApControl üst sınıfından OpenGLApControl ve DirectXApControl sınıfları türetilebilirdi.
  - Sonuçta benzer kodun birden fazla yerde tekrarlanması nedeniyle kusurlu bir tasarım ortaya çıkacaktı.
- **Gerçeklemelere göre değil, arayüzlere göre tasarlama yap.**
  - Böylece kara kutu yaklaşımını kullanabilirsin.
  - Örnekte ApControl sınıfı ComponentFactory sınıfından bir renderer ve bir shader oluşturmasını istiyor, bunu nasıl oluşturduğu umurunda değil.

Örnekte değişen şey hangi kod kütüphanesinin kullanılacağıydı. Bu görev ayrı bir yere verildi; değişen şey soyutlandı ve kara kutu yaklaşımıyla kullanıldı. Oyun motoru açısından iş mantığına daha çok odaklanılır ve kütüphanelerin kullanım ayrıntılarından mümkün olduğu kadar soyutlanılır.

Deneyimli tasarımcılar genelde kalıtım yerine parça-bütün ilişkisini tercih eder, çünkü kalıtımın yanlış kurulma olasılığı daha yüksektir ve durağan bir ilişki olduğu için yan etkileri daha ciddi olabilir. Bu örnekte her çözümde kalıtım kullanıldı; ders, kalıtımın doğru kurulması gerektiğidir. Kalıtımın tamamen yerine parça-bütün ilişkisinin kullanıldığı örnekler ileride gelecektir.

> **Ders sonrası not:** Hoca anlatırken "birinci çözümde kurduğumuz kalıtım hiyerarşisi bizi ters yerlere götürdü" dedi. Çözüm 1'de aslında kalıtım yoktu, sorun switch-case'ti. Slayttaki ifadeye göre kastedilen sakıncalı kalıtım, ApControl'den OpenGLApControl ve DirectXApControl alt sınıfları türetmek olurdu.

Arayüzlere göre tasarım yapmak kara kutu yaklaşımını mümkün kılar. Burada yine Java interface'i değil, public metotlar ve olası etkileşimler kastedilir. İlk çözümde belirli gerçeklemelerin belirli metotları üzerinden konuşuluyordu. İkinci ve üçüncü çözümlerde soyut üst sınıflar ve onlarda tanımlanan soyut metotlar vardı; bu türden nesne bekleyen yerlere ilgili somut nesne verildiğinde işler kendiliğinden çözüldü.

Hoca ayrıca şunu ekledi: ikinci ve üçüncü tasarımdaki soyut sınıfların üye alanı ve somut metodu yoksa, yalnız soyut metotları varsa, bunlar Java'da interface olarak da tanımlanabilir. Ama bu dile özgü bir çözüme girmek olur; hoca dersi mümkün olduğu kadar dilden bağımsız anlatmaya çalışır. Yalnız şemalar üzerinden de konuşulabilirdi, ama konuyu somutlaştırmak için kod yazmak gerekir; kod da daha geniş bir kitleye hitap etmek için dile özgü yapılardan kaçınılarak yazılır.

8. Ders Sonundaki Soru-Cevaplar

Soru: Tasarım kalıplarına daha fazla alışmak için önerebileceğiniz bir site ya da çalışma kaynağı var mı? **Cevap:** Ders notlarında ve ilk derste adı geçen kitaplara ek olarak tasarım kalıpları üzerine çok sayıda kitap vardır; bunlara kütüphanenin elektronik kaynakları üzerinden evden de ulaşılabilir. Bu tür kitaplar konuyu hep örnekler üzerinden anlatır. Hoca ayrıca Refactoring Guru (refactoring.guru) sitesini önerdi: hem tasarım kalıpları hem refactoring bölümleri vardır ve dersin tüm kapsamını içerir. Yaratımsal, yapısal ve davranışsal kalıpların hepsi listelenmiştir. Örneğin Abstract Factory sayfası, akılda kalan esprili grafiklerle bir problemden başlar ve sınıf tasarımını yavaş yavaş kurarak çözüme gider; sitedeki örnekler basılı kitap olarak da vardır. Hoca, benzer içeriklerin Medium yazılarında ve BTK Akademi'de de bulunabileceğini söyledi.

Soru: Nesneye dayalı analiz ve tasarım derslerinde genelde use case ve sequence diyagramları ağırlıklı oluyor. Bu derste de ilerleyen haftalarda bunlar üzerine mi yoğunlaşılacak? **Cevap:** Bu ders gereksinim analizi, analiz, tasarım, kodlama ve sınamanın bütün ayrıntılarını anlatan bir ders değildir; use case şemaları kesinlikle dersin kapsamında değildir. Genellikle sınıf

şemaları, bazen de sequence diyagramları kullanılacaktır; diğer UML şemalarına bu derste ihtiyaç yoktur. Dersin ana şablonu şöyledir: önce kalıp GoF kitabında anlatıldığı gibi sınıf şeması üzerinden anlatılır, sonra örnek bir problem verilir, bazen önce kod sonra şema, bazen tersi biçimde çözülür; ardından "şöyle bir değişiklik olacak, nasıl yaparız?" sorusuyla ilerlenir.

Soru (sohbetten): Yeni sınıfın tanımlanması oldukça büyük bir yük getiriyorsa, birinci çözümün daha etkili kabul edileceği bir durum oluşabilir mi? **Cevap:** Hayır. Yakın gelecekte mutlaka DirectX 11 gibi yeni kütüphaneler gelecektir. Yeni kütüphaneleri desteklememeye kesin karar verildiyse birinci çözümle devam edilebilir; zaten daha iyi bir tasarıma götüren şey farklı kütüphanelerin de eklenmesi ihtiyacıydı. Hoca "oldukça büyük yük"ün ne olabileceğini hayal etmeye çalıştı: kaynak kodu elde olmayan, yalnız ikili dosyaları olan bir yazılımı genişletmek ya da bileşenlerin ayrı sunucu çiftliklerinde, ayrı departmanların sistemlerinde çalıştığı durumlar. Bu durumlarda yeni bir şey eklememek avantajlı olabilir, ama bunlar dersin kapsamındaki konular değildir. Hoca bu örneklerin çok iyi olmadığını kabul etti ve daha somut örnek bulunursa paylaşılmasını istedi.

Örnek Sorular

Soru 1 (derste sözlü, 01:45:59): GoF'un WidgetFactory örneğindeki fabrika, Abstract Factory kalıbı yerine bir enum ve switch-case ile de kodlanabilir. Bu alternatifin nasıl çalıştığını ve dezavantajlarını açıklayın.

> **Çözüm (derste verildi):** Farklı aileleri simgeleyen bir enum tanımlanır, bir setter ile fabrikaya enum değeri verilir ve her create metodunda bütün ailelere case açılır. Kod uzar; asıl sorun yeni bir pencere yöneticisi geldiğinde enum'a değer eklemek, bütün switch'lere yeni case yazmak ve mevcut fabrika sınıfının kaynak kodunu değiştirmektir. Güncellemede bütün sınıflar yeniden dağıtılır. Abstract Factory'de ise yalnız yeni kardeş fabrika ve ürün sınıfları eklenir, mevcut koda dokunulmaz.

Soru 2 (ekrandan, 01:56:00 ve 02:08:00): Bir 3B oyun motoru, işletim sistemi ve ekran kartına göre OpenGL ya da DirectX kütüphanesinin renderer ve shader bileşenlerini kullanacak. Switch-case ile yazılmış çözümün zayıf yönü nedir? Kalıtım ve Abstract Factory ile nasıl iyileştirilir?

> **Çözüm (derste verildi):** Switch-case çözümünde yeni bir kütüphane (örneğin DirectX 11) eklemek için yeni sınıflar, enum'a yeni değer, ApControl'e yeni üye alanlar ve her işlemin switch'ine yeni case gerekir; yani çok fazla yerde kod değişir. Kalıtımlı çözümde soyut Renderer ve Shader sınıfları ve onların OpenGL/DirectX alt sınıfları kurulur; switch yalnız kurucuda kalır. Abstract Factory çözümünde soyut ComponentFactory (createRenderer, createShader) ile OpenGLFactory ve DirectXFactory tanımlanır, ApControl kurucuya fabrikayı alır ve ana sınıfta hiç switch kalmaz. Bedeli 3 ek sınıf ve her yeni kütüphane için bir ek fabrika sınıfıdır.

Soru 3 (derste sözlü, 02:22:23): Fabrika nesnesini kurucuya doğrudan verme numarası Çözüm 2'de (kalıtım ve çok biçimlilik) de yapılamaz mıydı?

> **Çözüm (derste verildi):** Tam olarak yapılamaz. Çözüm 2'de yalnız bir enum ve birbirinden bağımsız renderer ve shader nesneleri vardır; enum değerine bakıp hangi renderer ve shader'ın oluşturulacağını seçen bir switch-case ya da if-else if zinciri mutlaka gerekir. Çözüm 3'ün üstünlüğü, ilişkili nesnelerin oluşturulmasını tek bir fabrika nesnesinde toplayarak switch-case'i iş mantığı sınıflarından tamamen uzaklaştırmasıdır.

Soru 4 (derste sözlü, 02:37:14): Yeni sınıfın tanımlanması oldukça büyük bir yük getiriyorsa, switch-case'li birinci çözümün daha etkili kabul edileceği bir durum oluşabilir mi?

> **Çözüm (derste verildi):** Genel olarak hayır, çünkü yakın gelecekte yeni kütüphaneler mutlaka gelecektir ve esneklik ihtiyacını doğuran şey tam da budur. Yalnız yeni kütüphanelerin kesinlikle desteklenmeyeceğine karar verildiyse birinci çözümle devam edilebilir.

Soru 5 (ekrandan, 01:40:00): GoF kitabındaki WidgetFactory sınıf şeması UML standardına hangi açılardan uymaz? UML'ye uygun hâle getirmek için ne yapılmalıdır?

Çözüm (derste verildi): Soyut sınıf (WidgetFactory, Window, ScrollBar) ve soyut metotlar yalnız italik yazılarak gösterilmiştir; UML'de sınıf adının üstüne «`abstract`» stereotipi, soyut metotların yanına `{abstract}` yazılır. Metotların dönüş türü gösterilmemiştir; UML'de parantezden sonra iki nokta ile yazılır (örneğin `CreateScrollBar(): ScrollBar`). Kalıtım üçgeni alt sınıfların birleştiği yerde, üst sınıftan uzakta çizilmiştir ve karga ayağı gösterimine benzer; UML'de okun ucu üst sınıfa değecek biçimde çizilir.

Dersin Toparlaması

Bu derste tasarım kalıplarına giriş yapıldı. Tasarım kalıbı, yazılım tasarımında sık karşılaşılan sorunların yalın ve güzel çözümlerinin tarifidir; adı, sorunu, çözümü ve tartışma/sonuçlar bölümüyle dört bileşenden oluşur. Kalıplar tekerleği yeniden icat etmeden kaliteli, esnek ve yeniden kullanılabilir tasarım sağlar, deneyimi yazılı aktarır ve ekip içinde ortak bir sözlük kurar. Ama sihirli değnek değildirler; her çözümün güçlü ve zayıf yönleri tartılmalıdır. GoF kalıpları amaçlarına göre yaratımsal, yapısal ve davranışsal; odaklarına göre sınıfsal ve nesnesel olarak gruplanır.

İlk kalıp MVC'de veri, gösterim ve yönetim "değişeni bul ve sarmala" ilkesiyle Model, View ve Controller sınıflarına ayrıldı; Model'in hiçbir sınıfa bağımlı olmaması bağlaşımı en aza indirir. Bildirim oku, birden fazla View ve Controller gibi çeşitlemeler kalıbın ihtiyaca göre değiştirilebileceğini gösterdi. Yaratımsal kalıplardan Abstract Factory ise ilişkili nesne ailelerini somut sınıf adı vermeden oluşturur. Oyun motoru örneğinde switch-case, kalıtım ve Abstract Factory çözümleri karşılaştırıldı ve üç genel kural çıkarıldı: değişeni sarmala, kalıtım yerine parça/bütün ilişkisini tercih et, gerçeklemeye değil arayüze göre tasarla. Bir sonraki derste yaratımsal kalıplarla, Factory Method'dan başlayarak devam edilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.